UNIVERSITY OF RUHUNA

Faculty of Engineering

Department of Computer Engineering

EC4307 – Web Application Development

Mini Project Report

NEXZORA LMS

A Role-Based Learning Management System

Student name	NIRMAL M P L
Registration number	EG/2023/5761
Module	EC4307 – Web Application Development
GitHub repository	github.com/laskshithnirmal/NEXZORA-LMS
Date of submission	01 November 2026

Abstract

NEXZORA LMS is a web-based learning management system developed for the EC4307 Web Application Development mini project. The system has three user roles. Administrators manage students, teachers, classes, enrollments, fee payments and announcements; teachers publish PDF notes, lesson-video links and class announcements; and students register, join classes using an enrollment code, open learning materials and view their payment history. The frontend is a React 18 single-page application, and the backend is a Spring Boot 3.3 REST API secured with Spring Security and JSON Web Tokens (JWT). All data is stored in a MySQL 8 database of six related tables. Full create, read, update and delete (CRUD) operations were implemented and verified for users, classes and payments. The automated test suite passed (three tests, no failures), and a restart test confirmed that the data persists in MySQL.

1. Introduction

An educational institution has to manage classes, learning resources, enrollments, fees and notices. NEXZORA LMS combines these tasks in one web application in which each user sees only the functions that belong to their role. This report describes the requirements, architecture, database design, implementation, CRUD operations, validation, testing and limitations of the system. The description is based on the source code in the project repository and on tests of the running application.

1.1 Problem Statement

When class details, study materials, payment records and announcements are kept in separate places, the records easily become inconsistent. Administrators find it difficult to keep information up to date, teachers cannot easily limit materials to their own students, and students have no single place to find the information that concerns them. A secure, database-backed application with clearly separated user roles addresses these problems.

1.2 Aim and Objectives

The aim of the project was to build a full-stack web application with persistent storage and complete CRUD functionality. The specific objectives were to:

1. provide secure login and student self-registration, with role-based access for administrators, teachers and students;
2. implement full CRUD for the main entities (users, classes and payments) with the data stored in MySQL;
3. allow teachers to share PDF materials and lesson-video links, and allow students to join classes with an enrollment code;
4. validate input on both the client and the server and display clear error messages;
5. provide a consistent, responsive user interface and a documented, version-controlled code base.

1.3 Scope

The system covers three role-based portals, class and enrollment management, PDF and video learning materials, fee-payment records and announcements for all users or for a single class. Payments are recorded manually; there is no online payment gateway. The application runs locally, with the frontend on port 5173, the backend on port 8080 and MySQL on port 3306.

2. Technologies and System Requirements

Table 1 lists the technologies used. The versions were confirmed from `package.json`, `pom.xml` and the application logs.

Table 1: Technologies used in NEXZORA LMS

Layer	Technology	Purpose
Frontend	React 18.3, Vite 5.4, React Router 6.28	Single-page user interface, routing and development server
Styling / UI	Tailwind CSS 3.4, lucide-react, react-hot-toast	Responsive layout, icons and notification messages
HTTP client	Axios 1.7	REST requests; attaches the JWT to each request
Backend	Java 17, Spring Boot 3.3.5 (Web, Validation)	REST API and Jakarta Bean Validation
Security	Spring Security, JJWT 0.12.6, BCrypt	Stateless JWT authentication (HS256, 24 h) and password hashing
Persistence	Spring Data JPA, Hibernate 6.5, MySQL Connector/J	Object-relational mapping and repositories
Database	MySQL 8.0 (version 8.0.46 verified), InnoDB	Persistent storage (database `nexzora_lms`)
Build and test	Maven wrapper 3.9.9, JUnit 5, MockMvc, H2	Build and in-memory integration tests
Version control	Git, private GitHub repository	Source history and change tracking

Software requirements. Java 17 or later, Node.js 18 or later, MySQL 8 and a modern web browser. Maven does not need to be installed because the project includes the Maven wrapper. The functions available to each role are described in Section 5, and Table 3 in Section 6 summarises the CRUD operations.

3. System Architecture and Design

The system uses a three-tier client–server architecture (Figure 1). The React client sends JSON requests to `/api/*`, which the Vite development server forwards to the Spring Boot API. For each request, a JWT filter identifies the user and Spring Security checks the role: `/api/admin/**` requires ADMIN, `/api/teacher/**` requires TEACHER and `/api/student/**` requires STUDENT. Controllers validate the incoming data, services apply the business rules and ownership checks (a teacher can manage only their assigned classes, and a student can see only the classes they have joined), and Spring Data JPA repositories store the entities in MySQL. Uploaded PDF files are saved on disk under random UUID file names.

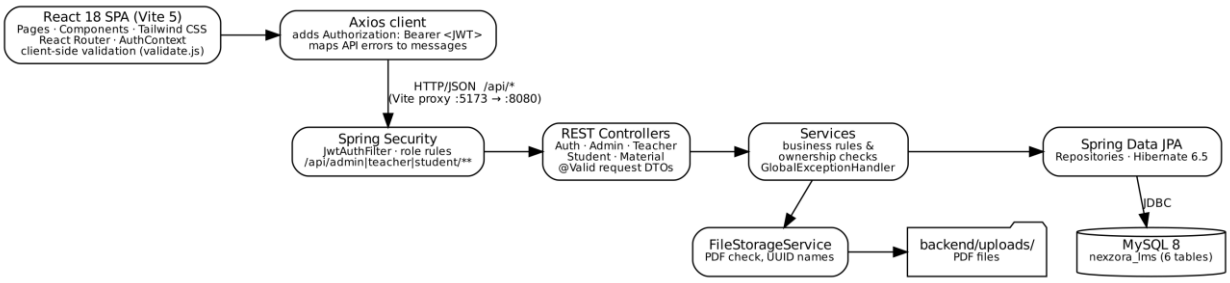


Figure 1: System architecture: React client, layered Spring Boot API, MySQL database and file storage

Code organisation. The backend is divided into the packages `config`, `controller`, `dto`, `entity`, `exception`, `repository`, `security`, `service` and `util` (40 Java source files). The frontend (34 source files) is divided into `api`, `components`, `context`, `hooks`, `pages` (with separate admin, teacher and student folders) and `utils`. Common interface elements, such as the modal window, confirmation dialog, search box and form field, are shared between pages.

Version control. The code is managed with Git and stored in a private GitHub repository (github.com/laskshithnirmal/NEXZORA-LMS). Each development stage is recorded as a separate commit. The `.gitignore` file excludes the `.env` file that holds secrets, `node_modules`, build output and uploaded files, so database passwords are not committed.

4. Database Design and ER Diagram

The `nexzora_lms` database is generated from the JPA entity classes (`ddl-auto=update`). The structure described here was read from the live MySQL 8.0.46 server on 31 October 2026 using `information_schema` and `SHOW CREATE TABLE`. The database contains six InnoDB tables with nine foreign-key constraints and three unique constraints (Table 2 and Figure 2).

Table 2: Tables, keys and relationships (verified in MySQL)

Table	Primary / unique keys	Foreign keys	Relationship
users	PK id; UK email	—	Stores administrators, teachers and students (role ENUM)
classes	PK id; UK enrollment_code	teacher_id → users.id (nullable)	A teacher teaches zero or more classes
enrollments	PK id; UK (student_id, class_id)	student_id → users.id; class_id → classes.id	Resolves the many-to-many relationship between students and classes
learning_materials	PK id	class_id → classes.id; uploaded_by → users.id	A class has zero or more PDF or video materials
announcements	PK id	author_id → users.id; class_id → classes.id (NULL = all users)	Announcements for all users or for one class
payments	PK id	student_id → users.id; class_id → classes.id (nullable)	A student has zero or more fee payment records

Data types. Every primary key is a `BIGINT AUTO_INCREMENT` column. Monetary values use `DECIMAL(10,2)` to avoid rounding errors. Roles, payment statuses and material types are stored as MySQL `ENUM` columns that correspond to Java enums. Timestamps use `DATETIME(6)`, payment dates use `DATE`, and text columns have length limits that match the validation rules (for example, `VARCHAR(120)` for e-mail addresses). Passwords are stored only as BCrypt hashes.

Integrity. Unique constraints prevent duplicate e-mail addresses, enrollment codes and enrollments. The foreign keys in MySQL do not cascade, so when a class or user is deleted, the service layer first removes or detaches the dependent rows within a single transaction. The reference script database/schema.sql reproduces this structure; on 31 October 2026 it was loaded into a scratch database and matched the live schema in all 49 columns.

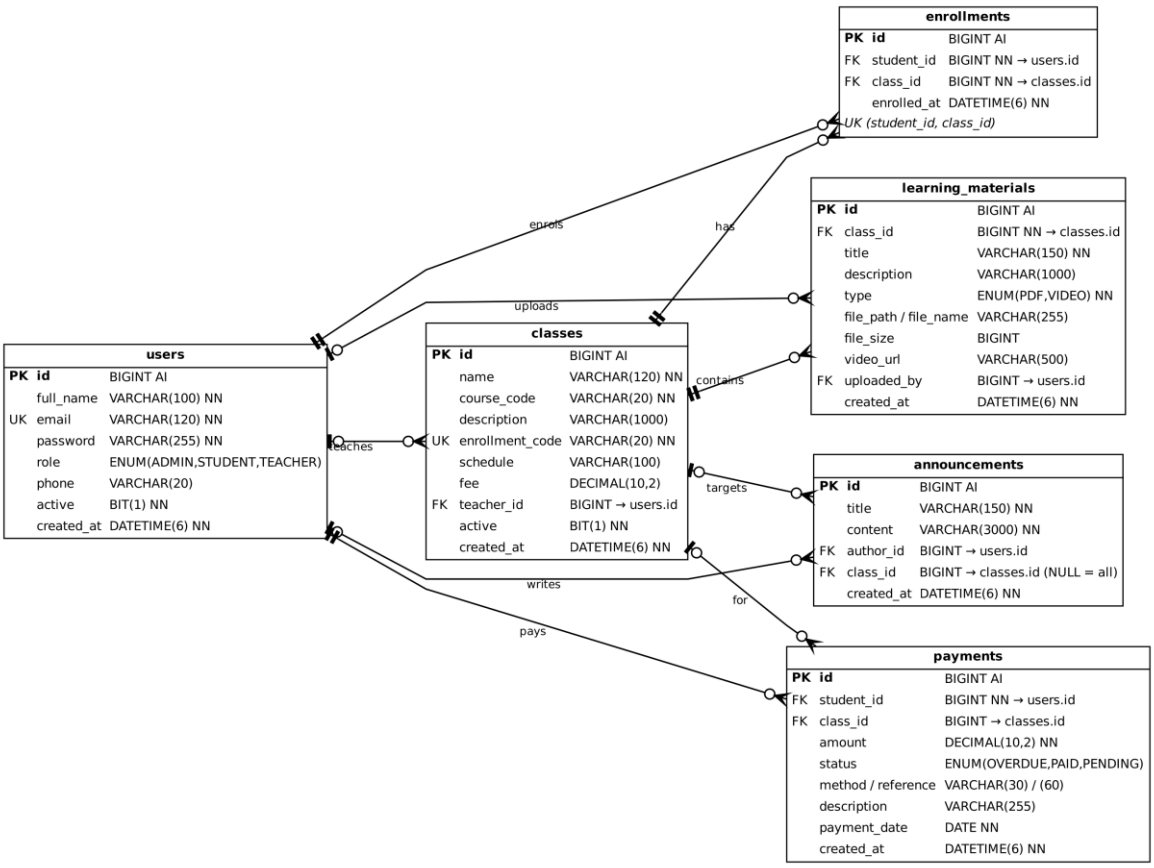


Figure 2: Entity–relationship diagram of the live nexzora_lms database (crow's-foot notation: ‖ exactly one, o| zero or one, o< zero or many)

5. System Implementation and Main Functionalities

The screenshots in this section were taken from the running application on 31 October 2026, after the interface redesign, while it was connected to the live MySQL database. Records named "Verify…" and the class "VER101" are temporary test data that were created for verification and deleted afterwards.

5.1 Authentication and Dashboards

Users sign in with their e-mail address and password. The backend returns a signed JWT, which the client stores and sends with every later request. Students can register themselves, whereas teacher accounts are created by the administrator. After signing in, each user is taken to the dashboard for their role (Figure 4).

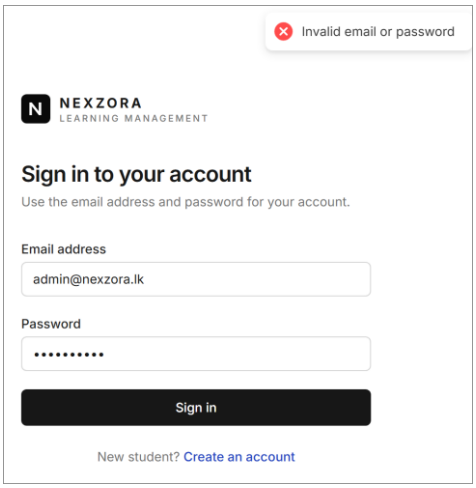


Figure 3: Login page showing the error returned for an incorrect password

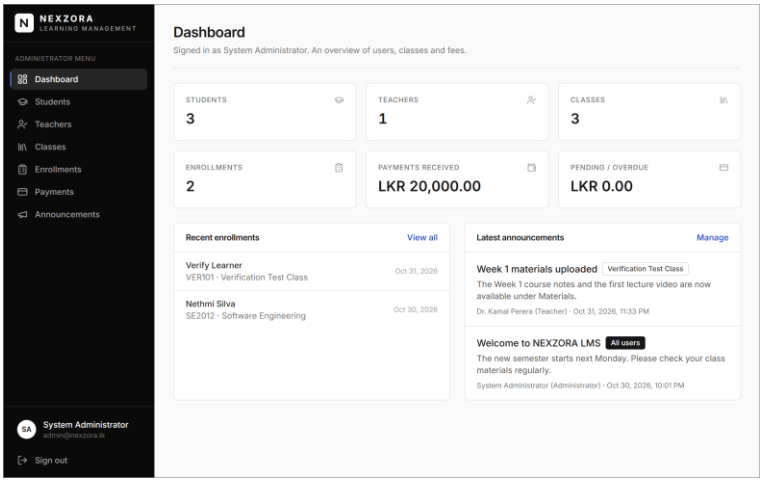


Figure 4: Administrator dashboard with live counts, payment totals and recent activity

5.2 User and Class Management

The administrator can create, edit, deactivate, search and delete student and teacher accounts (Figures 5 and 6). Each class has a name, course code, fee, schedule, optional teacher and enrollment code; if the code is left blank, the system generates one (Figures 7 and 8). The teacher of a class can also be changed directly from the class list.

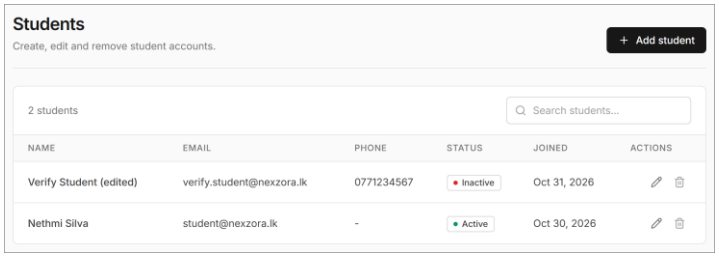


Figure 5: Student list after the update test (edited name and phone, status Inactive)

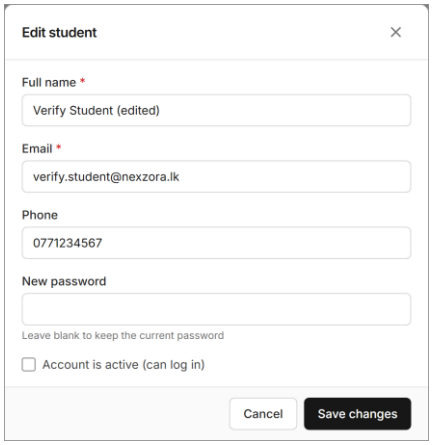


Figure 6: Edit-student form (password optional when updating)

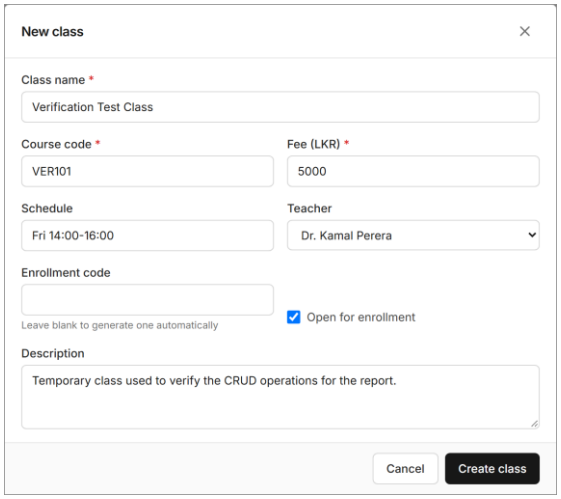


Figure 7: Create-class form with teacher assignment

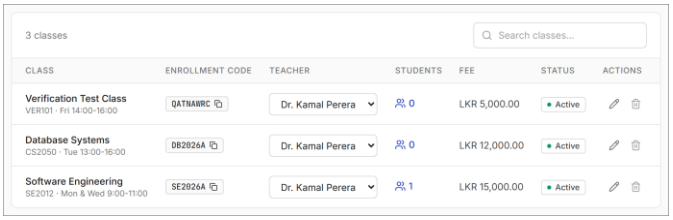


Figure 8: Class list showing the new class and its generated code (QATNAWRC)

5.3 Learning Materials, Enrollment, Payments and Announcements

Teachers can upload PDF files to their own classes and add YouTube or Vimeo lesson links; the server checks that each upload is a genuine PDF of at most 20 MB. A student joins a class by entering its enrollment code and can then view the class materials and announcements (Figure 9). PDF files are served only to authorised users. The administrator can view and remove enrollments (Figure 10), record and edit payments using a status filter and running totals (Figure 11; a payment can be linked only to a class in which the student is enrolled, and the form offers only those classes), and publish announcements to all users or to one class (Figure 12).

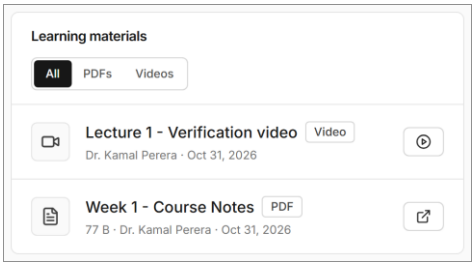


Figure 9: Student view of the class materials (PDF and video)

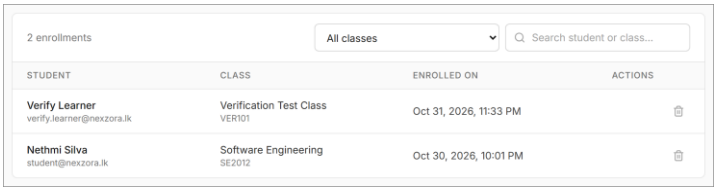


Figure 10: Enrollment list showing the student who joined with the code

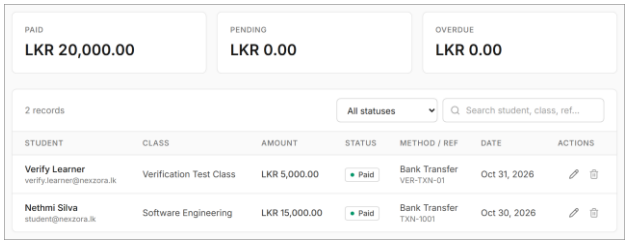


Figure 11: Payment management after the payment was updated to Paid

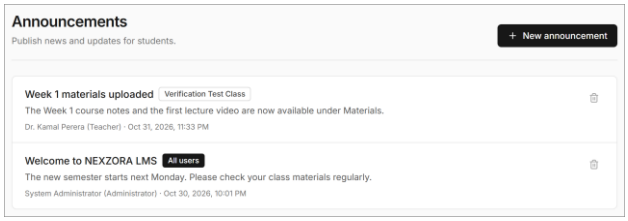


Figure 12: Announcements for one class (teacher) and for all users (administrator)

6. CRUD Functionality

Table 3 lists each CRUD operation, its REST endpoint and the result of testing it in the running application. Users, classes and payments support full CRUD, whereas enrollments, learning materials and announcements support create, read and delete only. Every delete action asks for confirmation and states what will be removed (Figure 13).

Table 3: CRUD operations verified in the running application (31 October 2026)

Entity	Create	Read (list / search / filter)	Update	Delete	Result
Users (student / teacher)	POST /api/admin/users; POST /api/auth/register	GET /api/admin/users?role= ; text search	PUT /api/admin/users/{id}	DELETE /api/admin/users/{id}	Pass (C, R, U, D)
Classes	POST /api/admin/classes	GET /api/admin/classes; search; student and teacher views	PUT /api/admin/classes/{id}; PUT …/{id}/teacher	DELETE /api/admin/classes/{id}	Pass (C, R, U, D)
Payments	POST /api/admin/payments	GET /api/admin/payments; search and status filter; GET /api/student/payments	PUT /api/admin/payments/{id}	DELETE /api/admin/payments/{id}	Pass (C, R, U, D)
Enrollments	POST /api/student/enroll (code)	GET /api/admin/enrollments; class filter	Not implemented	DELETE /api/admin/enrollments/{id}	Pass (C, R, D)
Learning materials	POST …/materials/pdf; …/materials/video	GET /api/{teacher|student}/classes/{id}/materials	Not implemented	DELETE /api/teacher/materials/{id}	Pass (C, R, D)
Announcements	POST /api/{admin|teacher}/announcements	GET /api/{role}/announcements	Not implemented	DELETE /api/{admin|teacher}/announcements/{id}	Pass (C, R, D)

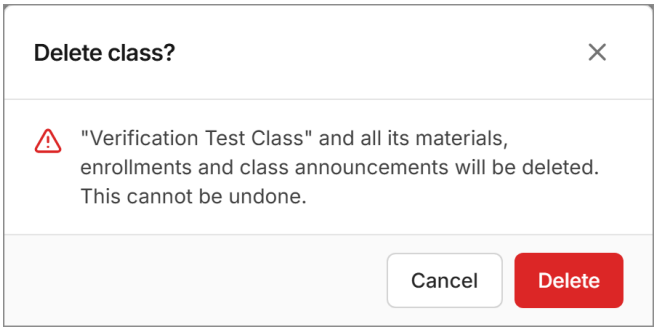


Figure 13: Delete confirmation listing the dependent data that will be removed

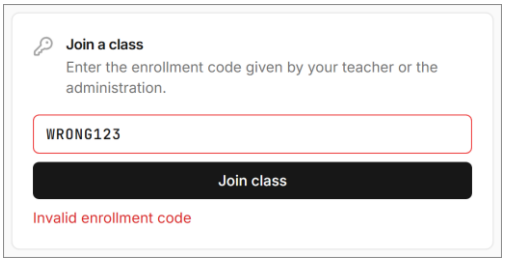


Figure 14: Error shown for an invalid enrollment code (HTTP 404)

Persistence test. With the test records in place (two students, one class, one PDF, one video, one announcement, one enrollment and one payment), the backend was stopped and restarted; the restart completed at 21:06:13. All records were still displayed in the application, and a direct MySQL query returned the edited student, the class with its updated fee (6500.00) and the payment with status PAID. All test records were then deleted through the user interface, which returned the database to its original contents. After the backend changes of 31 October 2026 the server was restarted again, and the existing records (two classes, one enrollment and the LKR 15,000 payment) were still returned by the API.

7. Validation and Error Handling

- **Client side.** The `validate.js` module checks required fields, e-mail and phone formats, length limits, numeric ranges, URL format and password confirmation before a request is sent, and it highlights each invalid field (Figure 15).
- **Server side.** Request DTOs are annotated with Jakarta Bean Validation constraints (`@NotBlank`, `@Email`, `@Size`, `@Pattern`, `@DecimalMin`, `@Digits`) and checked with `@Valid`. The services enforce business rules such as unique e-mail addresses and enrollment codes, the teacher role, active classes, ownership, no duplicate enrollments, and that a payment is linked only to a class in which the student is enrolled. Uploaded files must have a `.pdf` extension and begin with the `%PDF` signature.
- **Error responses.** A `@RestControllerAdvice` class converts exceptions into a consistent JSON response (`{status, message, errors}`) with the appropriate HTTP status code (400, 401, 403, 404, 409, 413 or 500). The client shows these as field messages or notifications (Figure 16) and returns the user to the login page when the token has expired.

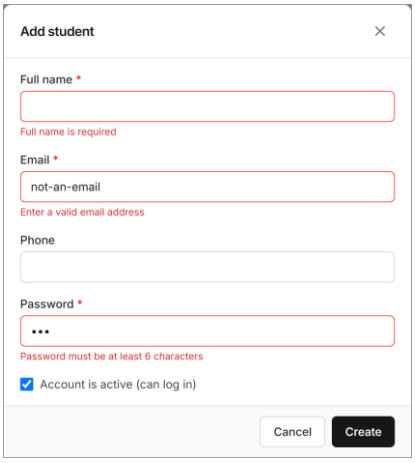


Figure 15: Client-side validation messages on the Add-student form

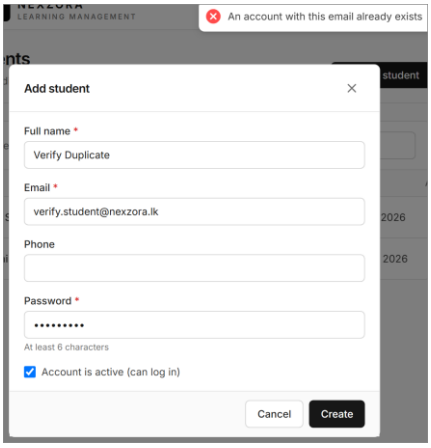


Figure 16: Server-side validation: duplicate e-mail rejected (HTTP 409)

8. Testing and Results

Automated tests. The project has one Spring Boot integration test class, `LmsIntegrationTest`, with three test methods. They call the real API through MockMvc with an in-memory H2 database. The first two cover login, registration validation, duplicate e-mail addresses, role-based access (401 and 403 responses), deactivated accounts, class creation and teacher assignment, enrollment by code, PDF upload and download, rejection of a non-PDF file, video links, announcement visibility, creating and updating payments, dashboards and the deletion of dependent records. The third, added on 31 October 2026, covers editing users and classes, an e-mail conflict on update, an administrator trying to delete their own account, a teacher trying to use another teacher's class (403), deleting materials, announcements, enrollments and payments, and the rule that a payment can be linked only to a class in which the student is enrolled. The suite was last run on 31 October 2026 with `mvnw.cmd test`: **3 tests run, 0 failures, 0 errors, BUILD SUCCESS** (Java 17, Windows 10), and the production frontend build (npm run build) completed without errors.

Manual testing. Every operation in Table 3 was tested in the running application using temporary data. After the interface redesign, the steps behind Figures 3–16 were repeated with a script that operated the application in a headless Chrome browser: it typed into the real forms and pressed their buttons (sign-in, add and edit student, create class, join class, edit payment and the delete confirmation). The same script created the test PDF, the video link, the class announcement and the pending payment through the REST API, because a headless browser cannot operate the operating-system file dialog. The error cases are shown in Figures 3, 14, 15 and 16. A separate check against the live MySQL database confirmed that a payment for a class is rejected (HTTP 400) before the student enrols and accepted (HTTP 201) afterwards, and that a student calling an administrator endpoint receives HTTP 403. Editing a class or user and deleting a payment, enrollment, material or announcement are now also covered by the automated tests.

9. UI/UX and Responsiveness

The interface uses one consistent black-and-white design. Black, white and neutral greys carry the layout; a single blue accent colour is reserved for links, keyboard-focus rings and the marker of the current page; red is used only for errors and delete actions; and small green, amber and red dots indicate payment and account status. Every page is built from the same shared components (page header, card, table, modal form, badge and statistic tile) defined in ui.jsx and index.css, and the NEXZORA "N" monogram is used for the logo and the browser icon. The layout uses Tailwind CSS responsive breakpoints. Below a width of 1024 px the black sidebar becomes a slide-in menu opened from a top bar; below 640 px the dashboard figures become compact rows, card grids collapse into one column, dialogs open as bottom sheets and wide tables scroll horizontally inside their card. Figures 4, 5, 8, 10, 11 and 12 show the desktop layout (1280 px browser window), and Figures 6, 9, 14 and 15 show the phone layout at a width of 500 px.

10. Installation and Setup

Full setup instructions are given in the repository README. In brief: (1) install Java 17 or later, Node.js 18 or later and MySQL 8; (2) copy `backend/.env.example` to `backend/.env` and set `DB_PASSWORD` and `JWT_SECRET`, or run `set-db-password.ps1`; (3) on Windows, run `start-all.cmd`, or run `mvnw spring-boot:run` in the `backend` folder and `npm install` followed by `npm run dev` in the `frontend` folder; and (4) open http://localhost:5173. The database and its tables are created automatically on the first start, together with a default administrator account and optional demo data.

11. Limitations and Future Improvements

- Enrollments, learning materials and announcements cannot be edited after they are created; edit forms could be added.
- Lists can be searched and filtered, but users cannot sort columns and there is no pagination. Sortable columns and server-side paging would improve this.
- Payments are entered manually; there is no online payment gateway or receipt generation.
- Uploaded PDF files are stored on the local disk, so cloud storage would be needed for a real deployment.
- The automated tests are three API-level integration tests; there are no unit or frontend tests yet.
- On phones, wide tables scroll horizontally inside their card instead of switching to a stacked card layout.
- There is no password-reset e-mail or token refresh, and administrator accounts cannot be managed through the interface.

12. Conclusion

NEXZORA LMS achieves the main objectives of the mini project. It is a working full-stack application with role-based security and a normalised MySQL database of six related tables. Full CRUD was verified for users, classes and payments, and create, read and delete operations were verified for enrollments, learning materials and announcements. The data remained in MySQL after a backend restart, input is validated on both the client and the server, and the automated tests passed. The limitations listed in Section 11 indicate the next steps for developing the system further.

References

[1] Spring Boot Reference Documentation. Available: https://docs.spring.io/spring-boot/

[2] Spring Security Reference. Available: https://docs.spring.io/spring-security/reference/

[3] Spring Data JPA Reference Documentation. Available: https://docs.spring.io/spring-data/jpa/reference/

[4] React Documentation. Available: https://react.dev/

[5] Vite Guide. Available: https://vite.dev/guide/

[6] Tailwind CSS v3 Documentation. Available: https://v3.tailwindcss.com/docs

[7] Oracle, MySQL 8.0 Reference Manual. Available: https://dev.mysql.com/doc/refman/8.0/en/

[8] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)", RFC 7519, IETF, 2015. Available: https://www.rfc-editor.org/rfc/rfc7519

[9] Jakarta Bean Validation 3.0 Specification. Available: https://jakarta.ee/specifications/bean-validation/3.0/

[10] JJWT – Java JWT library. Available: https://github.com/jwtk/jjwt